

**Design and Implementation of a
Linear Quadratic Regulator Controller for a
Quadrotor Linearised About Hover**

Henry Rochester

Supervisor: Dr Malcolm Hillebrand

Department of Mathematics and Applied Mathematics
University of Cape Town

May 3, 2026

Introduction

We now implement a Linear Quadratic Regulator (LQR) controller for the quadrotor. The objective of this controller is to drive the system to the hover equilibrium by minimising deviations in position, velocity, attitude, and angular velocity. This is achieved through the minimisation of a quadratic cost function.

Unlike the cascaded PID controller, which is constructed directly from error feedback terms, the LQR controller is derived from a model-based approach. We first linearise the nonlinear quadrotor dynamics about the hover equilibrium, and then solve for the optimal feedback gain matrix K which minimises the LQR cost objective.

The resulting control law is then applied to the error state, producing control input deviations required to drive the quadrotor toward the desired hover position in a stable and optimal manner.

Nonlinear Quadrotor State-Space Model

Recall that we define the quadrotor state vector as:

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

And the control input is given by:

$$\vec{u} = \begin{bmatrix} T & \tau_x & \tau_y & \tau_z \end{bmatrix}^T,$$

where T is the total thrust, and τ_x, τ_y, τ_z are the body torques about the roll, pitch, and yaw axes respectively.

The nonlinear system can be written compactly as:

$$\dot{\vec{x}} = f(\vec{x}, \vec{u}).$$

Hover Equilibrium

The hover equilibrium corresponds to the quadrotor remaining stationary at a fixed position with zero velocity, zero angular velocity, and level attitude. Thus, we write the equilibrium state as:

$$\vec{x}^* = \begin{bmatrix} x^* \\ y^* \\ z^* \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

At hover, we assume that the total thrust must balance the weight of the quadrotor:

$$T^* = mg.$$

Therefore, the equilibrium input is:

$$\vec{u}^* = \begin{bmatrix} mg & 0 & 0 & 0 \end{bmatrix}^T.$$

Linearisation About Hover

To design the LQR controller, we first linearise the nonlinear system about the hover equilibrium point, $(\vec{x}^*, \vec{u}^*)$.

We note that the objective of LQR is to drive the state and control-input errors, respectively, to zero, through the minimisation of the quadratic cost function. That is, once the drone reaches the desired hover position, we require our state-error, $\tilde{x} = 0$ and

control correction, $\tilde{u} = 0$. Thus, we define the perturbation variables:

$$\tilde{x} = \vec{x} - \vec{x}^*, \quad \tilde{u} = \vec{u} - \vec{u}^*.$$

The reason for this is that we cannot drive $\vec{x}$ and $\vec{u}$ themselves to zero, since this would either force the drone into the ground or result in zero thrust, causing it to fall out of the sky.

To linearise the system, we perform a first-order Taylor expansion about the equilibrium point, yielding:

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u},$$

where,

$$A = \left. \frac{\partial f}{\partial \vec{x}} \right|_{(\vec{x}^*, \vec{u}^*)}, \quad B = \left. \frac{\partial f}{\partial \vec{u}} \right|_{(\vec{x}^*, \vec{u}^*)}.$$

Thus, A is the Jacobian matrix describing how the state derivatives change with respect to each of the state variables, and B describes how the state derivatives change with respect to each of the control inputs. In this case, we have that $A \in \mathbb{R}^{12 \times 12}$ and $B \in \mathbb{R}^{12 \times 4}$.

For small roll and pitch angles (which is valid near hover), we use the following approximations:

$$\sin \phi \approx \phi, \quad \sin \theta \approx \theta, \quad \cos \phi \approx 1, \quad \cos \theta \approx 1, \quad \tan \theta \approx \theta,$$

which follow from first-order Taylor expansions, neglecting higher-order terms.

Under these assumptions, the nonlinear dynamics $\dot{\vec{x}} = f(\vec{x}, \vec{u})$ can be written as the following system of equations:

$$\begin{aligned}
\dot{x} &= v_x, \\
\dot{y} &= v_y, \\
\dot{z} &= v_z, \\
\dot{v}_x &= g \theta, \\
\dot{v}_y &= -g \phi, \\
\dot{v}_z &= -g + \frac{T}{m}, \\
\dot{\phi} &= \omega_x + \phi \theta \omega_y + \theta \omega_z, \\
\dot{\theta} &= \omega_y - \phi \omega_z, \\
\dot{\psi} &= \phi \omega_y + \omega_z, \\
\dot{\omega}_x &= \frac{1}{I_x} \left(\tau_x - (I_z - I_y) \omega_y \omega_z \right), \\
\dot{\omega}_y &= \frac{1}{I_y} \left(\tau_y - (I_x - I_z) \omega_z \omega_x \right), \\
\dot{\omega}_z &= \frac{1}{I_z} \left(\tau_z - (I_y - I_x) \omega_x \omega_y \right).
\end{aligned}$$

At hover, we assume $T = mg$, so that $\dot{v}_z \approx 0$, and all angular velocities are zero, since the drone would not be rotating during hover equilibrium.

To obtain the Jacobian matrix A , we compute:

$$A_{ij} = \left. \frac{\partial f_i}{\partial x_j} \right|_{(\vec{x}^*, \vec{u}^*)},$$

That is to say, we differentiate each component of f with respect to each state variable, and then evaluate at the hover equilibrium.

Doing so yields the following state matrix:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & g & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -g & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Similarly, the input matrix is obtained via:

$$B_{ij} = \left. \frac{\partial f_i}{\partial u_j} \right|_{(\vec{x}^*, \vec{u}^*)}.$$

Evaluating this at hover (where $T^* = mg$), we obtain:

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & \frac{1}{I_x} & 0 & 0 \\ 0 & 0 & \frac{1}{I_y} & 0 \\ 0 & 0 & 0 & \frac{1}{I_z} \end{bmatrix}.$$

Derivation of the Optimal LQR State-Feedback Controller

Now that we have found our Jacobian matrices A and B , we seek the control correction, $\tilde{u}$ which will minimise our LQR cost objective function:

$$J = \int_0^\infty (\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u}) dt,$$

where Q and R are state and control weighting matrices respectively, which determine how much each component of the state and control input is penalised. Moreover, we require Q to be symmetric and positive semi-definite, and R to be symmetric and positive definite. An intuitive reason for this is that if $R = 0$, then there is no penalty on the control input, meaning we could apply arbitrarily large inputs, which is not physically realistic. Moreover, the symmetry is what allows us to obtain our Algebraic Riccati Equation.

Now, we seek $\tilde{u}$, which is the change in control input required to minimise this cost objective function J . The way we do this is by defining a control law of the form:

$$\tilde{u} = -K\tilde{x}.$$

The way we arrive at this is as follows:

We begin by defining the optimal value function:

$$V(\tilde{x}) = \tilde{x}^\top P \tilde{x} = \min_{\tilde{u}} J = \min_{\tilde{u}} \int_0^\infty (\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u}) dt.$$

for some symmetric, positive-definite matrix P . The optimal value function can be interpreted as a measure of how poor the current state is, given that we act optimally from now on. That is, we choose $\tilde{u}$ in order to minimise the total remaining cost.

Now, the Hamilton-Jacobi-Bellman equation (HJB) is given by:

$$\min_{\tilde{u}} \left(\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u} + \frac{dV}{d\tilde{x}}^\top \dot{\tilde{x}} \right) = 0.$$

This means that we seek $\tilde{u}$ which minimises the instantaneous cost plus the rate of change of the value function, i.e. the current cost plus the future cost. The reason that we use HJB, is to ensure we are solving a global optimisation problem over time, and thus we include the future cost component.

We then compute:

$$\frac{dV}{d\tilde{x}} = 2P\tilde{x},$$

and substituting in $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$, we obtain:

$$\min_{\tilde{u}} (\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u} + (2\tilde{x}^\top P)(A\tilde{x} + B\tilde{u})) = 0.$$

Now, extracting only the terms which depend on $\tilde{u}$, we have:

$$\tilde{u}^\top R \tilde{u} + 2\tilde{x}^\top P B \tilde{u},$$

which we seek to minimise. Thus, taking the derivative with respect to $\tilde{u}$ and setting it equal to zero, we obtain the following expression:

$$2R\tilde{u} + 2B^\top P\tilde{x} = 0.$$

Dividing through by 2, we have that:

$$R\tilde{u} + B^\top P\tilde{x} = 0.$$

Thus, rearranging, we have:

$$\tilde{u} = -R^{-1}B^\top P\tilde{x}.$$

Letting

$$K = R^{-1}B^\top P,$$

we obtain the desired expression:

$$\tilde{u} = -K\tilde{x}.$$

Thus, K is our feedback controller gain. That is, given our current state error $\tilde{x}$, we compute $-K\tilde{x}$, which yields the optimal control correction that steers the drone towards our hover equilibrium position.

Algebraic Riccati Equation

Now the next logical question to ask is: how do we compute P ?

Returning to the Hamilton-Jacobi-Bellman equation, we have that the expression inside the minimisation is:

$$\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u} + (2\tilde{x}^\top P)(A\tilde{x} + B\tilde{u}) = 0.$$

Expanding this out, we obtain:

$$\tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u} + 2\tilde{x}^\top P A \tilde{x} + 2\tilde{x}^\top P B \tilde{u} = 0.$$

Substituting in our optimal control law $\tilde{u} = -K\tilde{x}$, we get:

$$\tilde{x}^\top Q \tilde{x} + \tilde{x}^\top K^\top R K \tilde{x} + 2\tilde{x}^\top P A \tilde{x} - 2\tilde{x}^\top P B K \tilde{x} = 0.$$

Now recalling that $K = R^{-1}B^\top P$, we obtain:

$$\tilde{x}^\top Q \tilde{x} + \tilde{x}^\top P B R^{-1} B^\top P \tilde{x} + 2\tilde{x}^\top P A \tilde{x} - 2\tilde{x}^\top P B R^{-1} B^\top P \tilde{x} = 0.$$

Thus, the control-related terms simplify, yielding:

$$\tilde{x}^\top Q \tilde{x} - \tilde{x}^\top P B R^{-1} B^\top P \tilde{x} + 2\tilde{x}^\top P A \tilde{x} = 0.$$

Now, note that $\tilde{x}^\top P A \tilde{x}$ is a scalar, and hence we can write:

$$\tilde{x}^\top P A \tilde{x} = (\tilde{x}^\top P A \tilde{x})^\top = \tilde{x}^\top A^\top P \tilde{x}.$$

Thus, we can write:

$$2\tilde{x}^\top P A \tilde{x} = \tilde{x}^\top (P A + A^\top P) \tilde{x}.$$

Substituting this in, we obtain:

$$\tilde{x}^\top Q \tilde{x} - \tilde{x}^\top P B R^{-1} B^\top P \tilde{x} + \tilde{x}^\top (P A + A^\top P) \tilde{x} = 0.$$

Since, Q , $P B R^{-1} B^\top P$ and $P A + A^\top P$ are all symmetric, we can rewrite the expression as:

$$\tilde{x}^\top (Q + A^\top P + P A - P B R^{-1} B^\top P) \tilde{x} = 0.$$

For this to hold for all state errors $\tilde{x}$, we require:

$$Q + A^\top P + P A - P B R^{-1} B^\top P = 0.$$

This is the Continuous Algebraic Riccati Equation (CARE), which we solve for P .

The CARE may yield multiple solutions for P . However, our objective is to stabilise the drone. Thus, for each candidate P , we compute:

$$K = R^{-1} B^\top P,$$

and substitute $\tilde{u} = -K\tilde{x}$ into the linearised system:

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u} = (A - BK)\tilde{x}.$$

We then select the solution P such that $(A - BK)$ is stable, which means all eigenvalues satisfy:

$$\text{Re}(\lambda) < 0.$$

Once P is obtained, we compute the feedback gain $K = R^{-1}B^\top P$.

Thus, the overall LQR procedure is as follows: we define our equilibrium hover state, compute the state error $\tilde{x}$, solve the CARE for P , compute the gain K , and then at each time step apply:

$$\tilde{u} = -K\tilde{x}.$$

Finally, to recover the actual control input, we add this correction to the equilibrium control input:

$$\vec{u} = \vec{u}^* + \tilde{u}.$$

Initial Choice of Q and R

We recall that the matrices Q and R represent the state and control weighting matrices respectively. As with the PID controller, these matrices must be tuned. However, before beginning the tuning process, it is important to initialise Q and R with reasonable values so that the tuning procedure is both efficient and meaningful.

The standard approach for doing this is to use Bryson's Rule.

We begin by choosing both Q and R to be diagonal matrices. From the LQR cost function, recall that we have:

$$x^\top Qx + u^\top Ru,$$

we see that the term $x^\top Qx$ penalises deviations in the state, while $u^\top Ru$ penalises control effort. The diagonal entries determine how strongly each individual state (or control input) is penalised, while the off-diagonal terms would introduce coupling between components.

For simplicity and better interpretability, we assume independence between the different state and control components, and thus restrict Q and R to be diagonal. This aligns with the assumption underlying Bryson's Rule, which is that the components are indeed independent.

Bryson's Rule prescribes that each diagonal entry of Q and R be chosen as:

$$Q_{ii} = \frac{1}{x_{i,\max}^2}, \quad R_{jj} = \frac{1}{u_{j,\max}^2},$$

where $x_{i,\max}$ and $u_{j,\max}$ represent the maximum allowable deviations for the state and control components respectively.

The motivation for this choice is to make the cost function dimensionless and properly scaled. That is to say, without normalisation, variables with larger physical units could be penalised more heavily, even if they are less important for system stability. For example, a small angular deviation may be far more critical than a small positional error.

To see this explicitly, we define the normalised state as:

$$x_i^{\text{norm}} = \frac{x_i}{x_{i,\max}}.$$

Then the state cost becomes:

$$x^\top Q x = \sum_i Q_{ii} x_i^2 = \sum_i Q_{ii} x_{i,\max}^2 (x_i^{\text{norm}})^2.$$

By choosing $Q_{ii} = \frac{1}{x_{i,\max}^2}$, we obtain:

$$x^\top Q x = \sum_i (x_i^{\text{norm}})^2,$$

which shows that we are penalising the squared normalised deviation of each state. An identical argument holds for the control cost $u^\top R u$.

Using this principle, we initialise Q and R as follows:

$$Q = \text{diag} \left(1.0, 1.0, 4.0, 1.0, 1.0, 4.0, 33.0, 33.0, 8.0, 4.0, 4.0, 4.0 \right),$$

$$R = \text{diag} \left(0.11, 1.0, 1.0, 2.0 \right).$$

These values correspond to the following design choices:

- Position errors: x, y are limited to 1 m, while z to 0.5 m.
- Velocity errors: 1 m/s for x, y and 0.5 m/s for z .
- Attitude errors: roll and pitch are constrained to approximately 10° , while yaw is allowed up to 20° for initial tuning (note that we do use radians, but explain in

degrees for ease of visualisation).

- Angular velocities: limited to approximately 0.5 rad/s.
- Control inputs: thrust deviation limited to 3 N (30% of hover thrust), while torques are constrained to roughly 20% of their maximum achievable values.

These initial values are intentionally somewhat lenient. The goal at this stage is not optimal performance, but rather to provide a well-scaled starting point from which tuning can be performed.

Tuning Procedure

The tuning procedure follows a similar philosophy to that of the PID controller, but now operates through the weighting matrices Q and R rather than explicit gains.

We begin by isolating the vertical dynamics. Specifically, we introduce small perturbations in z and v_z . This is a natural starting point, since near hover the vertical motion is governed by:

$$\ddot{z} = -g + \frac{T}{m},$$

and thus depends almost entirely on the thrust input.

We then tune the entries Q_{22} and Q_{55} (corresponding to z and v_z respectively) to achieve a response that is both fast and well-damped. The objective is to ensure that the drone reaches the desired altitude quickly, without oscillations, and without overshooting or undershooting the target position.

Once satisfactory vertical behaviour is obtained, we move to the attitude dynamics. We repeat the same procedure for roll (ϕ, ω_x) , and pitch (θ, ω_y) , respectively, where we tune the corresponding diagonal entries of Q to stabilise the angular motion and ensure a fast, non-oscillatory response.

With stable attitude control established, we then proceed to horizontal motion. The states (x, v_x) and (y, v_y) are tuned next, since we recall that horizontal motion is coupled to attitude (through pitch and roll), so effective control here relies on the previously tuned roll and pitch dynamics.

Finally, we tune the yaw dynamics (ψ, ω_z) . Yaw is treated last, as it is the least critical component for stability in the hover regime.

Once the state weighting matrix Q has been tuned to produce the desired system response, we perform a final refinement step on the control weighting matrix R . Adjusting R allows us to balance the control effort with responsiveness, thus ensuring that the system reaches the target rapidly while reducing any oscillations and avoiding overly aggressive control inputs.

References

- [1] J. P. Hespanha, *Lecture Notes on LQR/LQG Controller Design*, February 27, 2005.
- [2] B. Douglas, *Understanding LQR Control*, YouTube. Available at: https://www.youtube.com/watch?v=E_RDCF01Jx4
(Accessed: April 2026).
- [3] C. Lum, *Introduction to Linear Quadratic Regulator (LQR) Control*, YouTube. Available at: <https://www.youtube.com/watch?v=wEevt2a4SKI>
(Accessed: April 2026).
- [4] Z. M. Manaa, A. M. Abdallah, M. A. Abido, and S. S. A. Ali, *Koopman-LQR Controller for Quadrotor UAVs from Data*, June 2024.
- [5] M. T. Asfaw et al., *Modeling and Control Design of Quadrotor UAV using LQR Controller with Integral Action*, January 2024.
- [6] Stanford University, *EE363: Continuous-Time Linear Quadratic Regulator Lecture Notes*, Winter 2008-2009.